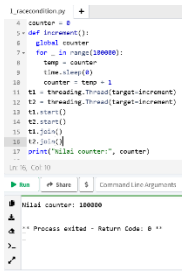
# FORMAT LAPORAN PRAKTIKUM

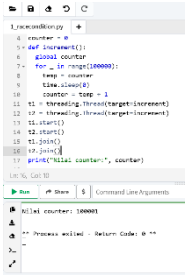
A. PRAKTIKUM 1 - Simulasi Race Condition

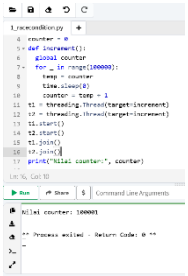
1. Tujuan dari praktikum ini adalah:
   - Memahami konsep thread dalam sistem operasi.
   - Mengamati secara langsung terjadinya race condition.
   - Menganalisis critical section pada program multithreading.
2. Screenshot hasil eksekusi :

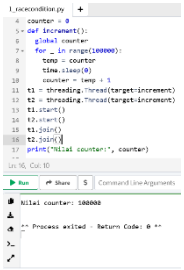
   Program dijalankan menggunakan dua thread yang mengakses variabel global counter.

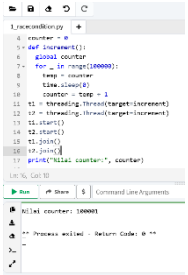
   Hasil yang diperoleh setelah beberapa kali menjalankan program:




3. Analisis dari setiap percobaan :

   Secara teori:

   100000 + 100000 = 200000

   Namun hasil sering lebih kecil karena:
   - Operasi increment tidak atomik.
   - Terjadi interleaving instruksi.
4. Identifikasi masalah concurrency :

   Terjadi ketika beberapa thread mengakses data yang sama tanpa sinkronisasi.
5. Perbandingan teknik sinkronisasi :
   1. Tanpa Sinkronisasi, mudah tetapi tidak aman.
   2. Mutex (Mutual Exclusion), digunakan untuk mengunci critical section sehingga hanya satu thread yang dapat mengakses data pada satu waktu. Paling umum diigunakan untuk melindungi critical section.
6. Kesimpulan Konseptual :
   - Shared Resource: Thread berbagi ruang memori yang sama, sehingga variabel global counter dapat diakses oleh banyak thread sekaligus.
   - Race Condition: Terjadi karena beberapa thread memodifikasi data bersama tanpa proteksi, sehingga hasil akhir bergantung pada timing eksekusi yang tidak teratur.
   - Operasi Non-Atomik: Kesalahan nilai (hasil < 200.000) disebabkan karena instruksi counter += 1 terbagi menjadi beberapa langkah yang bisa disela oleh thread lain (interleaving).
   - Kegagalan Integritas: Tanpa mekanisme sinkronisasi, validitas data dalam sistem multithreading tidak dapat dijamin meski secara teori perhitungannya sudah benar.

- Kebutuhan Mutual Exclusion: Diperlukan pembatasan akses (critical section) agar hanya satu thread yang dapat memodifikasi data pada satu waktu untuk menjaga konsistensi.

B. PRAKTIKUM 2 - Perbaikan Dengan Mutex

1. Tujuan dari praktikum ini adalah:
   - Mengimplementasikan mutual exclusion menggunakan objek Lock pada Python.
   - Memahami cara kerja mutex dalam melindungi shared resource dari gangguan interleaving thread.
   - Menjamin konsistensi data pada variabel global dalam lingkungan multithreading.
2. Screenshot hasil eksekusi :

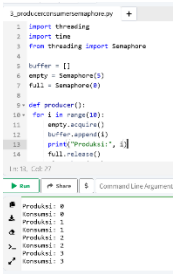
3. Analisis dari setiap percobaan :
   - lock menjamin hanya satu thread masuk critical section.
   - Mutual exclusion tercapai.
   - Hasil selalu konsisten : Karena akses dilakukan secara bergantian (satu per satu), tidak ada instruksi yang terpotong di tengah jalan, sehingga 100.000 iterasi dari t1 dan 100.000 iterasi dari t2 terjumlah sempurna menjadi 200.000.
4. Identifikasi masalah concurrency :
   - Race Condition (Kondisi Balapan): Masalah di mana beberapa thread mencoba menulis ke memori yang sama di waktu yang hampir bersamaan.
   - Non-Atomic Operation: Operasi counter += 1 di Python sebenarnya melibatkan tiga langkah: membaca nilai, menambah satu, dan menyimpan kembali. Tanpa mutex, thread bisa "disela" di antara langkah-langkah ini.
   - Data Inconsistency: Tanpa sinkronisasi, nilai akhir variabel global tidak dapat diprediksi dan sering kali salah.
5. Perbandingan teknik sinkronisasi :
   - Kelebihan: Sangat efektif untuk perlindungan data sederhana dan menjamin mutual exclusion yang ketat.
   - Kekurangan: Jika tidak hati-hati (misalnya lupa melepas kunci atau urutan kunci yang salah), dapat menyebabkan deadlock di masa depan.

- Karakteristik: Hanya membolehkan satu thread untuk mengakses critical section pada satu waktu tertentu.

6. Kesimpulan Konseptual :
   Mutex adalah solusi fundamental dalam pemrograman multithreading untuk mengatasi race condition. Dengan menerapkan mekanisme penguncian, kita memastikan syarat Mutual Exclusion terpenuhi, di mana integritas data tetap terjaga meskipun banyak thread bekerja secara bersamaan pada sumber daya yang sama.

C. PRAKTIKUM 3 - Producer Consumer (Semaphore)

1. Tujuan dari praktikum ini adalah:
   - Mengimplementasikan koordinasi antar thread menggunakan Semaphore.
   - Memahami mekanisme penghitungan slot kosong (empty) dan item tersedia (full) pada buffer.
   - Mencegah terjadinya masalah overflow (buffer penuh) dan underflow (buffer kosong).
2. Screenshot hasil eksekusi :




   Log akan menunjukkan proses "Produksi" dan "Konsumsi" yang berjalan bergantian secara harmonis. Jika buffer penuh, produsen akan menunggu; jika buffer kosong, konsumen akan menunggu.

3. Analisis dari setiap percobaan :
   - Inisialisasi Semaphore: empty diset 5 (kapasitas buffer) dan full diset O (belum ada barang).
   - Peran Produsen: Menggunakan empty.acquire() untuk memastikan ada slot kosong sebelum menambah data ke buffer. Setelah menambah data, ia memanggil full.release() untuk memberi tahu konsumen bahwa ada barang baru.
   - Peran Konsumen: Menggunakan full.acquire() untuk menunggu hingga ada barang tersedia. Setelah mengambil data, ia memanggil empty.release() untuk menambah jumlah slot kosong yang bisa diisi kembali oleh produsen.
   - Sinkronisasi: Penggunaan semaphore memastikan produsen tidak memproduksi lebih dari 5 item sekaligus dan konsumen tidak mengambil barang dari buffer kosong.
4. Identifikasi masalah concurrency :
   - Inter-process Communication (IPC): Masalah bagaimana dua thread saling memberi sinyal kapan harus bekerja dan kapan harus menunggu.
   - Buffer Overflow: Risiko produsen terus mengisi data saat memori sudah penuh.
   - Buffer Underflow: Risiko konsumen mencoba mengambil data saat buffer tidak memiliki isi.
5. Perbandingan teknik sinkronisasi :

- Semaphore: Bertindak sebagai "penghitung" (counter) yang memungkinkan koordinasi kompleks antar thread (seperti sinkronisasi urutan kerja).
- Mutex (Praktikum 2): Lebih sederhana, hanya untuk mengunci akses satu per satu agar tidak terjadi tabrakan data (mutual exclusion).

6. Kesimpulan Konseptual :
   Semaphore sebagai Sinyal: Semaphore bukan sekadar pengunci, melainkan mekanisme pemberi sinyal status sumber daya (penuh/kosong).
   Koordinasi Thread: Teknik ini menjamin alur kerja yang sinkron antara produsen dan konsumen sehingga penggunaan memori (buffer) menjadi efisien dan aman.
   Manajemen Antrean: Semaphore sangat efektif untuk mengelola akses pada sumber daya yang memiliki kapasitas terbatas (lebih dari satu).

D. PRAKTIKUM 4 – Simulasi dan Perbaikan Deadlock

1. Tujuan dari praktikum ini adalah:
   - Mengidentifikasi terjadinya deadlock dalam lingkungan multithreading.
   - Menganalisis kondisi circular wait yang menyebabkan sistem berhenti beroperasi.
   - Mengimplementasikan teknik resource ordering untuk memperbaiki dan mencegah deadlock.
2. Screenshot hasil eksekusi :

   Pada Simulasi Deadlock: Program akan terlihat "hang" atau berhenti tanpa mencetak kata "selesai" karena kedua thread saling menunggu. Sedangkan, Pada Perbaikan Deadlock: Program akan berjalan lancar dan mencetak "Thread 1 selesai" serta "Thread 2 selesai" secara berurutan seperti gambar di samping.

3. Analisis dari setiap percobaan :
   - Kondisi Awal (Deadlock): Thread 1 mengunci lock1 dan menunggu lock2, sementara Thread 2 mengunci lock2 dan menunggu lock1. Karena keduanya memegang satu sumber daya dan menunggu sumber daya lainnya, terjadi kemacetan total.
   - Mekanisme Perbaikan: Perbaikan dilakukan dengan menyamakan urutan pengambilan kunci (resource ordering).
   - Hasil Perbaikan: Dengan membuat Thread 2 juga mengambil lock1 terlebih dahulu sebelum lock2, risiko saling menunggu secara melingkar dapat dihilangkan, sehingga sistem dapat menyelesaikan tugasnya.
4. Identifikasi masalah concurrency :
   - Mutual Exclusion: Sumber daya hanya bisa digunakan satu thread.
   - Hold and Wait: Thread memegang satu sumber daya sambil menunggu yang lain.

- No Preemption: Sumber daya tidak bisa diambil paksa dari thread yang memegangnya.
- Circular Wait: Terjadi rantai saling menunggu antar thread.

5. Perbandingan teknik sinkronisasi :
    - Tanpa Aturan (Deadlock): Thread mengambil sumber daya secara acak atau tidak konsisten, memicu risiko circular wait.
    - Resource Ordering (Solusi): Menetapkan hierarki atau urutan global untuk semua sumber daya, sehingga tidak akan pernah terjadi siklus saling menunggu.
6. Kesimpulan Konseptual :
    - Deadlock adalah Macet Total: Kondisi ini lebih berbahaya daripada race condition karena program benar-benar berhenti berfungsi.
    - Pentingnya Urutan: Masalah sinkronisasi bukan hanya tentang "mengunci" data, tetapi juga tentang "bagaimana" dan "kapan" kunci tersebut diambil secara konsisten.
    - Pencegahan Berbasis Aturan: Deadlock dapat dihindari dengan merancang alur program yang memastikan salah satu kondisi Coffman (terutama circular wait) tidak pernah terpenuhi.